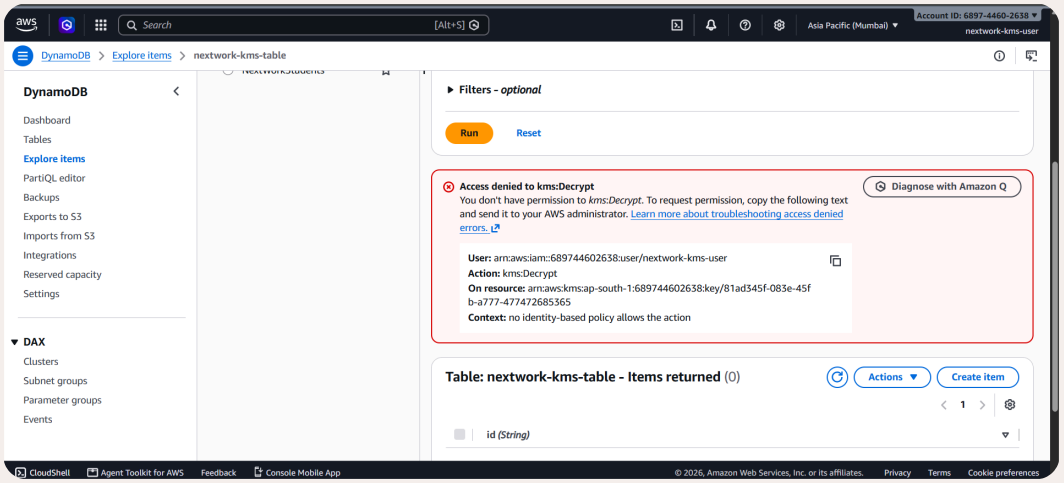
ENCRYPT DATA WITH AWS KMS

By Ahmed Umar Rehman | October 2026



Introducing Today's Project!

The goal is to learn how AWS Key Management Service (KMS) can be used to encrypt and protect data

Tools and concepts

Services I used include AWS KMS, Amazon DynamoDB, and AWS IAM.
Key concepts I learnt include encryption, KMS keys, key policies, IAM permissions, encryption at rest, and decryption.

Project reflection

This project took me approximately 1 hour. The most challenging part was understanding how KMS key policies and IAM permissions work together. It was most rewarding to see how granting KMS permissions allowed the test user to decrypt and access the protected DynamoDB data.

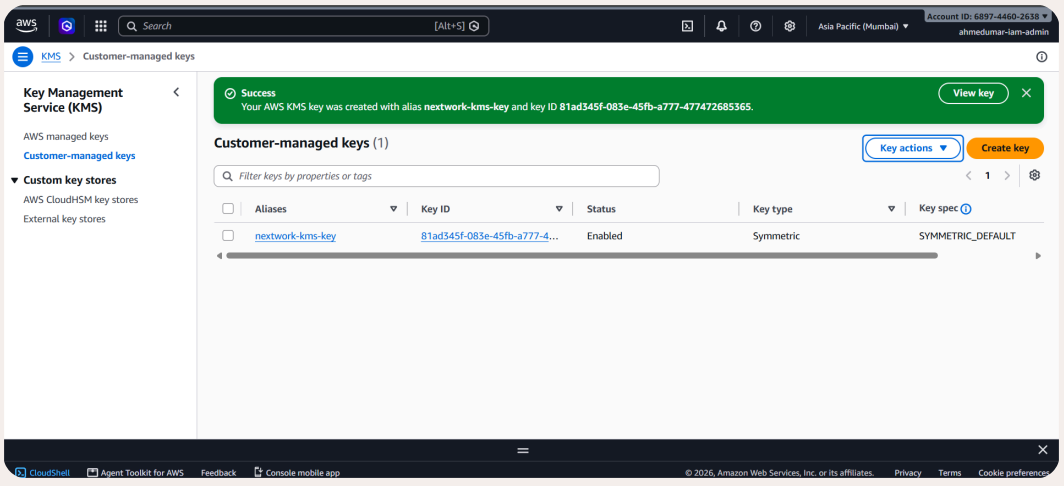
I chose to do this project today because I wanted to understand how AWS KMS protects data and how encryption works with other AWS services.
Something that would make learning with NextWork even better is more detailed guidance on IAM permissions and common permission errors.

Encryption and KMS

Encryption is the process of converting readable data (plaintext) into unreadable data (ciphertext) using an encryption key.
Companies and developers do this to protect sensitive data from unauthorized access.
Encryption keys are secret pieces of information used to encrypt and decrypt data.

AWS KMS is a managed AWS service that lets you create, manage, and control encryption keys used to protect your data.
Key management systems are important because they securely manage encryption keys and control who can use them to protect sensitive data.

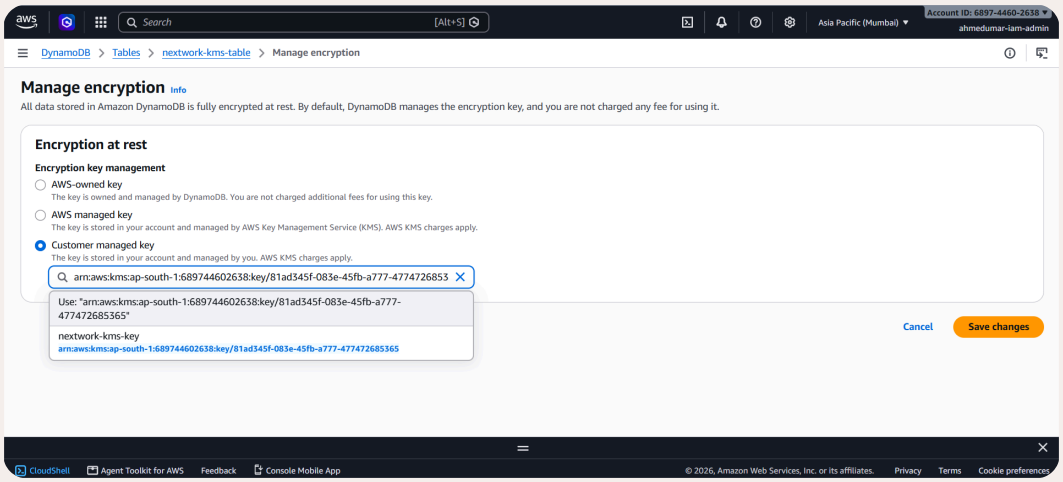
Encryption keys are broadly categorized as symmetric keys and asymmetric keys.
I set up a symmetric key because the same key is used to encrypt and decrypt the data, making it suitable for efficiently protecting data within AWS.



Encrypting Data

My encryption key will safeguard data in DynamoDB, which is a managed AWS NoSQL database service used to store and retrieve data.

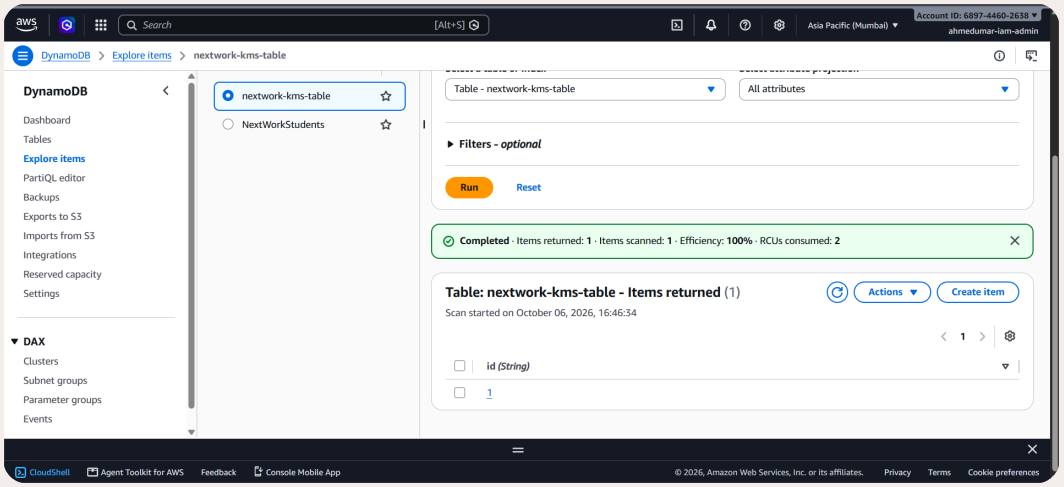
The different encryption options in DynamoDB include AWS owned keys, AWS managed keys, and customer managed keys.
Their differences are based on who owns and manages the encryption key and how much control you have over it.
I selected a customer managed key because it gives me more control over the encryption key through AWS KMS.



Data Visibility

Rather than controlling who has access to the key, KMS manages user permissions by using key policies and IAM policies to control who can use the key and what actions they can perform.

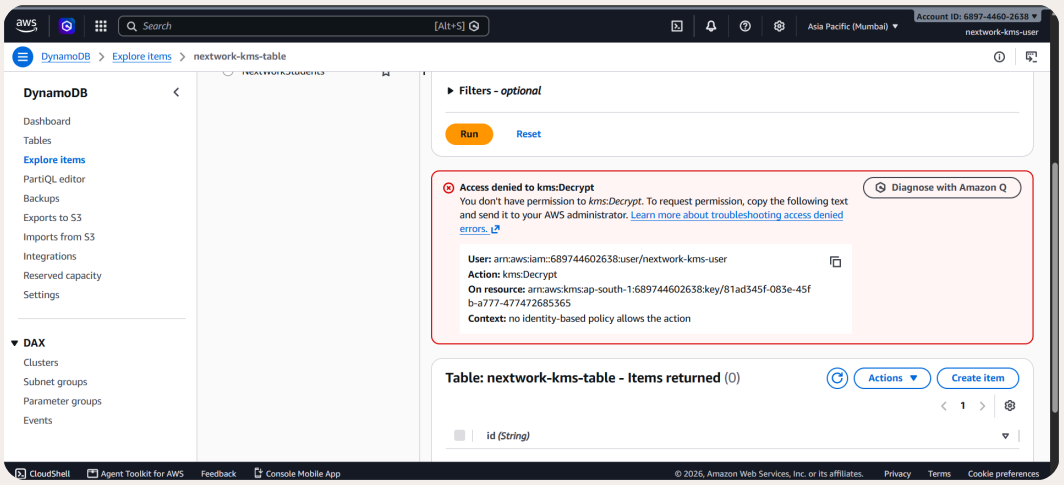
Despite encrypting my DynamoDB table, I could still see the table's items because DynamoDB automatically decrypts the data for authorized users when they access it.
DynamoDB uses transparent data encryption, which means data is automatically encrypted when stored and decrypted when accessed by an authorized user, without requiring the application to manage the encryption process.



Denying Access

I configured a new IAM user to access and manage DynamoDB data.
The permission policies I granted this user are AmazonDynamoDBFullAccess but not permissions to use the KMS encryption key.

After accessing the DynamoDB table as the test user, I encountered an access denied error because the user did not have permission to use the KMS encryption key.
This confirmed that access to the DynamoDB table alone is not enough to access data protected by a customer managed KMS key.



EXTRA: Granting Access

To let my test user use the encryption key, I added the test user as a principal in the KMS key policy.
My key's policy was updated to allow the test user to use the key for cryptographic operations such as decrypting data.

Using the test user, I retried viewing the DynamoDB table's items.

I observed that I could now access the items, which confirmed that the test user had permission to use the KMS key and decrypt the encrypted data.

Encryption secures data instead of controlling who can access it.
I could combine encryption with IAM permissions to protect data and control who can access and use it.

